SCHEME AND EXCEPTIONS Meta

COMPUTER SCIENCE MENTORS 61A

November 11 – November 14, 2025

Example Timeline

- Ice Breaker [3 min]

  If you want, start off with an icebreaker of your choice!

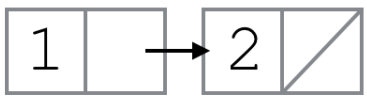
  Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!
- Mini-lecture: Scheme Cond, Begin, Lists, and Symbolic Programming [10 min]
- Q1: WWSD - Lists [7 min]
- Q2: WWSD - Define [8 min]
- Q3: Six-Sevens [7 min]
- Q4: Waldo [7 min]
- Mini-Lecture: Python Exceptions [7 min]
- Q5: List Sum [7 min]

1 Scheme

1. What will Scheme output? Draw box-and-pointer diagrams to help determine this. (Ask your mentor if you're unsure what's going on. You aren't expected to understand this completely on your own.)

```
scm> (cons 1 (cons 2 nil))
```

```
(1 2)
```



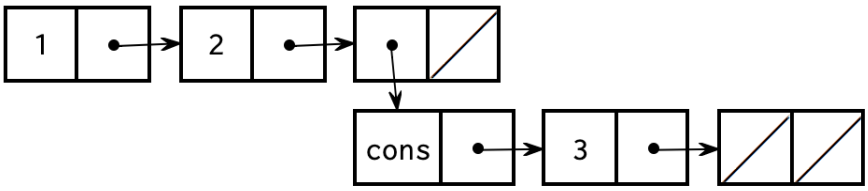

```
scm> (cons 1 '(2 3 4 5))
```

```
(1 2 3 4 5)
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

When we use the quote before the list, we are saying that we should put the literal list (2 3 4 5) in the cdr of this list. So in this case we create a list where the first element (car) is 1, and the cdr is the list (2 3 4 5).

```
scm> (cons 1 '(2 (cons 3 nil)))

(1 2 (cons 3 ()))
```



Since we also used a quote here, we do not evaluate the `(cons 3 nil)`. We keep everything inside the quotes the same so the `cdr` of this list is the list `(2 (cons 3 nil))`. That means that we add the element 2, and then the nested list `(cons 3 nil)`.

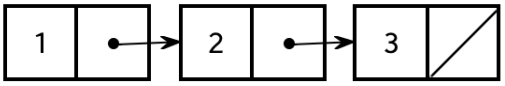
```
scm> (cons 1 (2 (cons 3 nil)))

eval: bad function in : (2 (cons 3 nil))
```

While evaluating the operands, Scheme will try to evaluate the expression `(2 (cons 3 nil))`. Since 2 is not a valid operator, this expression Errors.

```
scm> (cons 3 (cons (cons 4 nil) nil))

(3 (4))
scm> (define a '(1 2 3))

a
```

Defines a list of elements of `(1 2 3)` and binds the list to the variable `a`. Recall that define returns the name of the symbol.

```
scm> a

(1 2 3)
```



```
scm> (car a)

1

scm> (cdr a)

(2 3)

scm> (car (cdr a))

2
```

From above, we know that `(cdr a)` is `(2 3)`. From that, we can evaluate `(car (cdr a))` to 2.

How can we get the 3 out of a?

```
(car (cdr (cdr a)))
```

To get to the pair that contains 3, we need to call `(cdr (cdr a))`. To get the element 3, we need the `car` of `(cdr (cdr a))`.

Teaching Tips

- Draw diagrams or use the 61A Scheme Web interpreter for visualizing lists
- Encourage students to ask questions and experiment with extra `cons`, `car`, and `cdr` statements to see how they change the outputs of statments!
- While unrelated to the problem, it may be helpful to teach students these keywords:
  - `(pair? arg)`, which checks if arg has a first and rest
  - `(list? arg)`, which returns true if arg is a well-formed list

Suggested Time: 7 min

2. What will Scheme output?

```
scm> (define c 4)

c

scm> ((define (x) 1))

Error: str is not callable: x

scm> (x)

1

scm> ((lambda (x y) (+ c)) 1 2)

4

scm> (eval 'c)

2

scm> '(cons 1 nil)

(cons 1 nil)

scm> (eval (list 'if '(even? c) 1 2))

1

scm> (let ((a (+ 3 1)) (b 3)) (+ a b) (/ a b))

1.333333333333333
```

Teaching Tips

- In the first four questions, we can see that defining a function using the define keyword and then calling it on the same line does not work but defining a function using the lambda keyword and then calling it on the same line does work. This is because `(define (x)1)` evaluates to x and then Scheme is trying to apply/call the string x, however you cannot apply a string, you have to apply a procedure. This question teaches us essentially how Scheme works for call expressions. All the expressions in the list are evaluated and then Scheme tries to call the evaluated expression of the first expression with the evaluated expressions of the rest of the expressions in the list as parameters.
- Quotation marks are easily one of the trickiest concepts in Scheme, so spend a lot of time making sure your students understand these problems thoroughly! It would help to do a quick mini lecture or review of quotation marks if your students need.
- In some cases it is easier to think of the single quotation mark as double quotes in Python that encompass a string, such as in "standalone" expressions like the third one. In such cases the exact "string" is returned
- Encourage students to make the connection between Scheme lists and Scheme expressions as often as they can. Being able to read Scheme expressions as lists will be very helpful in the future.
- Whenever there is an eval with a quote, you go down one level of evaluation and essentially pretend the quote is not there.
- Going off of the third point, it could be beneficial to model Scheme expressions on a pyramid of evaluation: exact string, evaluating the string, etc.
- While doing the one with the list operator, go to the Scheme Specification and the other links and show them how to navigate through the pages.

Suggested Time: 8 min

3. Define `six-sevens`. Return the number of times that `7` follows `6` in the list.

```
> (six-sevens '(4 6 7 6 0 7))
1
> (sixty-ones '(7 6 7 4 6 7 6 0 7))
2
> (sixty-ones '(6 7 6 7 4 6 7 6 0 7))
3
```

```
(define (six-sevens lst)
    (cond (_____________________________________________)
          (_____________________________________________________________________________)
          (else _____________________________)))
```

```
(define (six-sevens lst)
    (cond ((or (null? lst) (null? (cdr lst))) 0)
          ((and (= 6 (car lst)) (= 7 (car (cdr lst)))) (+ 1 (six-sevens
             (cdr(cdr lst)))))
          (else (six-sevens (cdr lst)))))
```

Encourage students to think about the logic and solutions in Python first if they seem stuck. Considering developing a Python version of solutions with the students first and walk through 'translating' the

solution to Scheme together.

Suggested Time: 7 min

4. Implement `waldo`. `waldo` returns `#t` if the symbol waldo is in a list.

```
scm> (waldo '(1 4 waldo))
#t
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f
```

```
(define (waldo lst))
```

```
(define (waldo lst)
    (cond ((null? lst) #f)
          ((eq? (car lst) 'waldo) #t)
          (else (waldo (cdr lst)))
      )
  )
```

Alternate solution:

```
(define (waldo lst)
    (if (null? lst)
        #f
        (if (eq? (car lst) 'waldo)
            #t
            (waldo (cdr lst))
        )
    )
)
```

Similar to how we focus just think about first and rest when working with linked lists, we want to think about solving this question in terms of car and cdr in scheme. At a high level, we can check we can check (car lst) is equal to waldo. If it is, we can return true. Otherwise, we check the rest of the list: (cdr lst). Last, we conclude that waldo is not in the list when we have checked every element but still not found at match.

Observe that the second doctest is essentially the simplest input we can give. From here, we get our first base case– if we are given an empty Scheme list, then return `#f`. Otherwise, notice that regardless of where in the list we find 'waldo', our function should return `#t`. So our function can just stop after we find the first instance of 'waldo in the list, if it exists. Therefore, our second base case checks if our current element (the car of lst) is 'waldo and returns `#t` if this is true. Finally, if neither base case is satisfied, then we must search through the rest of lst, excluding the first element we just checked, for the string 'waldo. So, we make a recursive call using the cdr of lst.

Many problems that involve traversing through a Scheme list will have a similar structure to this solution. Recursive calls on (cdr lst) are also common, similar to recursive calls on link.rest. Note that you can also write a solution using two nested ifs instead of cond, but cond is often a little cleaner to read.

Teaching Tips

- Waldo is good for a warm-up question, but try to do the challenge problem afterwards!
- Emphasize the recursive nature of Linked Lists and Scheme functions
- Make sure students are comfortable with accessing the "first" and "rest" of lists in Scheme
- Teach the difference between = (only works for numbers) and `eq?`
- Note we need to use the quote syntax `'waldo` to compare the string to the values in our list
  - Without the quote, we get an error because the symbol `waldo` is undefined

5. **Extra challenge:** Define `waldo` so that it returns the index of the list where the symbol waldo was found (if waldo is not in the list, return `#f`).

```
scm> (waldo '(1 4 waldo))
2
scm> (waldo '())
#f
scm> (waldo '(1 4 9))
#f

(define (waldo lst)









)
```

```
(define (waldo lst)
    (define (helper lst index)
        (cond ((null? lst) #f)
              ((eq? (car lst) 'waldo) index)
              (else (helper (cdr lst) (+ index 1)))
        )
    )
    (helper lst 0)
  )
```

Recall HW2, problem 3, "pingpong" without assignment. To get around the assignment restriction, we introduced a new variable to keep track of state. We passed some state into a helper function to keep track of our iteration. In this problem, we will use the same idea for our implementation.

Inside our recursive helper function, we use the same logic as the original waldo question. Specifically, if the list is empty, we return false. If the first element we see is equal to waldo, we now return the index we have been tracking instead of true. Finally, to recursively search, we call the helper function on the rest of the list and increment the index by 1, returning the result of that call. We call this helper function initially with arguments 'lst' and index 0 to start from the beginning of the list.

A logical alternate solution would be to increment 1 after the recursive call to helper (without incrementing index), instead of adding one to the index, and then recursing. This solution would actually be incorrect. When we increment by one, if the element is not found, `#f` would not be properly returned. At the end of recursion, we would try to add 1 to `#f`, which results in an error.

Teaching Tips

- Discuss with students the different ways to keep track of an extra variable index with a recursive function
  - Explain how the standard approach of keeping track of an index and using iteration is impossible with Scheme
  - Guide students to the conclusion that we can keep track of the extra information during recursion by creating a helper function

Suggested Time: 7 min

2 Exceptions

1. `list_sum` is a function that adds up the elements of a list and returns the sum. However, we have mixed types in our list! Write `list_sum` such that rather than using any **`if`** or **`for`** statements, it uses exceptions to catch these errors.

```
def list_sum(lst):
    '''
    >>> list_sum([1, 2, 3, 4, 5])
    15
    >>> list_sum([1, '2', 3, '4', 5])
    9
    >>> list_sum(['1', '2', 3, 4, 5])
    12
    '''
    i = 0
    total = 0
    while True:
        try:
            total = total + lst[i]
        except IndexError:
            return total
        except TypeError:
            i += 1
            continue
        i += 1
```

Teaching Tips

- Write out a version with exception handling and ask students what type of errors will be raised:
  - Not allowing **`if`** statements: checking for `TypeError` allows us to determine when we have a non-integer value.
  - Not allowing **`for`** statements: checking for `IndexError` allow us to determine when we've reached the end of a list (and are going out of bounds).
- Remind students on how to use **`try`** and **`except`** statements, as well as how to catch only a specific type of error.

Suggested Time: 6 min